

CS 550 Operating Systems

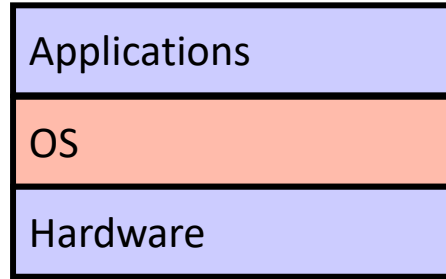
Spring 2026

Course overview

What happens when a program runs?

- A running program executes instructions, where the processor **fetches** an instruction from memory, **decodes** it, and **executes** it. Then processor moves on to the next instruction, and so on, and so on, until the program finally completes (i.e., the Von Neumann computing model).
- There is a body of software to make it easy to run programs, allowing programs to share memory, enabling programs to interact with devices, and other useful features.
 - That body of software is called the **operating system** (another early name for the OS was the **supervisor**).

What is an OS?



- A software layer between the hardware and the application programs/users which provides a **virtual machine interface**.
- A resource manager that
 - allows **concurrent** programs/users to share the hardware resources, and
 - ensures information stored **persistently** in the computer.
- Three themes of operating systems: **virtualization**, **concurrency** and **persistence**.

Theme 1: Virtualization

- Virtualize underlying hardware to make it more powerful and much easier to use for **different programs** running in the OS.
 - the OS takes a physical resource (such as the processor, or memory, or a disk) and transforms it into a more general, powerful, and easy-to-use virtual form of itself.
- A different level of virtualization, which virtualizes hardware for **different operating systems**, will be introduced later in the semester if time permits.

Virtualization example – Virtualizing the CPU

cpu.c (simple code that loops and prints)

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int
main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\n");
        exit(1);
    }
    char *str = argv[1];
    while (1) {
        Spin(1);
        printf("%s\n", str);
    }
    return 0;
}
```

Virtualization example – Virtualizing the CPU

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
A
^C
prompt>
```

Run one instance of “cpu” program on a single core CPU

The CPU runs the program and outputs to the terminal the messages that the program wants to print.

Virtualization example – Virtualizing the CPU

```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
```

```
[1] 7353
```

```
[2] 7354
```

```
[3] 7355
```

```
[4] 7356
```

```
A
```

```
B
```

```
D
```

```
C
```

```
A
```

```
B
```

```
D
```

```
C
```

```
A
```

```
C
```

```
B
```

```
D
```

```
...
```

Run many instances of “cpu” program on a single core CPU

*Even though we have only one processor, somehow all four of these programs seem to be running at the same time! **How?***

*The **operating system**, with some help from the hardware, turns (or virtualizes) a single CPU (or small set of them) into a seemingly infinite number of CPUs and thus allowing many programs to seemingly run at once → virtualizing the CPU.*

Virtualization example – Virtualizing Memory

```
1  #include <unistd.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "common.h"
5
6  int
7  main(int argc, char *argv[])
8  {
9      int *p = malloc(sizeof(int));
10     assert(p != NULL);
11     printf("(%d) address pointed to by p: %p\n",
12           getpid(), p);
13     *p = 0;
14     while (1) {
15         Spin(1);
16         *p = *p + 1;
17         printf("(%d) p: %d\n", getpid(), *p);
18     }
19     return 0;
20 }
```

Q1: Where is the variable `p` stored?

Q2: Where is the memory returned by `malloc()` stored?

Virtualization example – Virtualizing Memory

Run a single instance of the “mem” program.

```
prompt> ./mem
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

Run two instances of the “mem” program.

```
prompt> ./mem & ./mem &
[1] 24113
[2] 24114
(24113) address pointed to by p: 0x200000
(24114) address pointed to by p: 0x200000
```

Are the addresses pointed to by the pointer p (i.e., the buffer addresses returned by the malloc() function calls) in the two processes the same or different?

Virtualization example – Virtualizing Memory

Run a single instance of the “mem” program.

```
prompt> ./mem
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

Run two instances of the “mem” program.

```
prompt> ./mem & ./mem &
[1] 24113
[2] 24114
(24113) address pointed to by p: 0x200000
(24114) address pointed to by p: 0x200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
(24113) p: 4
(24114) p: 4
...
```

For the two instances case, each running program has allocated memory at the same address (0x200000), and yet each seems to be updating the value at 0x200000 independently! It is as if each running program has its own private memory, instead of sharing the same physical memory with other running programs.

Virtualization example – Virtualizing Memory

Run a single instance of the “mem” program.

```
prompt> ./mem
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

Run two instances of the “mem” program.

```
prompt> ./mem & ./mem &
[1] 24113
[2] 24114
(24113) address pointed to by p: 0x200000
(24114) address pointed to by p: 0x200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
(24113) p: 4
(24114) p: 4
...
```

Virtualizing memory:

- *Each process accesses its own private virtual memory address space (sometimes just called its address space), which the OS somehow maps onto the physical memory of the machine.*
- *A memory reference within one running program does not affect the address space of other processes (or the OS itself); as far as the running program is concerned, it has physical memory all to itself.*

Theme 2: concurrency

- **Virtualization** allows multiple concurrent programs to share the virtualized hardware resources, which brings out another class of topics: **concurrency**.
- The problems of concurrency arose first within the operating system itself (as we see in the previous virtualizing CPU example).
- The problems of concurrency are no longer limited just to the OS. Modern multi-threaded programs exhibit the same problems.

Concurrency Example

```
volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}

int
main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);

    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value   : %d\n", counter);
    return 0;
}
```

Note:

1. The global variable "counter" is shared among all the threads in the process.
2. The keyword "volatile" tells the compiler that the "counter" variable may change at any time. The compiler must reload the variable whenever it references it rather than relying upon a copy it might have in a processor register.

Concurrency Example

Expected output?

```
prompt> gcc -o thread thread.c -Wall -pthread
prompt> ./thread 1000
Initial value : 0
Final value   : 2000
```

Reality?

```
prompt> ./thread 100000
Initial value : 0
Final value   : 143012    // huh??
prompt> ./thread 100000
Initial value : 0
Final value   : 137298    // what the??
```

What happened?

A key part of the program above, where the shared counter is incremented, takes three instructions: one to load the value of the counter from memory into a register, one to increment it, and one to store it back into memory. Because these three instructions do not execute **atomically** (all at once), strange things can happen.

Theme 3: Persistence

- In system memory, data can be easily lost, as devices such as DRAM store values in a volatile manner; when power goes away or the system crashes, any data in memory is lost. Thus, we need hardware and software to be able to store data persistently.
- The hardware comes in the form of some kind of input/output or I/O device (e.g., hard drives, printers, displays).
- The software in the operating system that usually manages the disk is called the file system; it is thus responsible for storing any files the user creates in a reliable and efficient manner on the disks of the system.